

ТЕХНИЧЕСКИЙ СПРАВОЧНИК

3x-ui и Xray

Соединения, протоколы и конфигурация

От сетевых уровней и DPI до inbound, routing, DNS и balancers

Версия справочника	0.11
Срез интерфейса	3x-ui v3.6.0
Xray-core	v26.7.28
Дата проверки	20 августа 2026 года

Оффлайн-версия. Названия полей сохранены в формулировках панели.

Содержание

Крупные разделы справочника. В Word названия работают как внутренние ссылки.

О документе

1. Карта соединения: что находится на каком уровне
2. Прокси-протоколы: VLESS, Trojan и Hysteria2
3. TLS, REALITY, SNI, ECH, ALPN и uTLS
4. Sniffing: что он видит и зачем нужен
5. DPI: базовые принципы и классификация прокси-трафика
6. Создание inbound: полный каталог полей 3x-ui v3.6.0
7. Xray Configs: что это за раздел
8. Balancers — подробное руководство
9. DNS в Xray Configs
10. Advanced
11. IPv6, Anycast, rotating IP, load balancers и cloud edge
12. Xray и реально используемые альтернативные ядра
13. Практические профили и проверка
14. Быстрый указатель полей со скриншотов
15. Термины, которые нельзя смешивать
16. Основные источники

О документе

Этот справочник объясняет соединение от нижних сетевых уровней к настройкам 3x-ui, а затем последовательно разбирает создание inbound и раздел **Xray Configs**. Названия полей оставлены такими, как в панели, а рядом дано русское назначение.

Справочник основан на исходном коде и документации стабильного релиза 3x-ui v3.6.0, а также на актуальной документации Xray-core. Перечень полей дополнительно сверен с пользовательскими скриншотами; сами скриншоты в документ не включены.

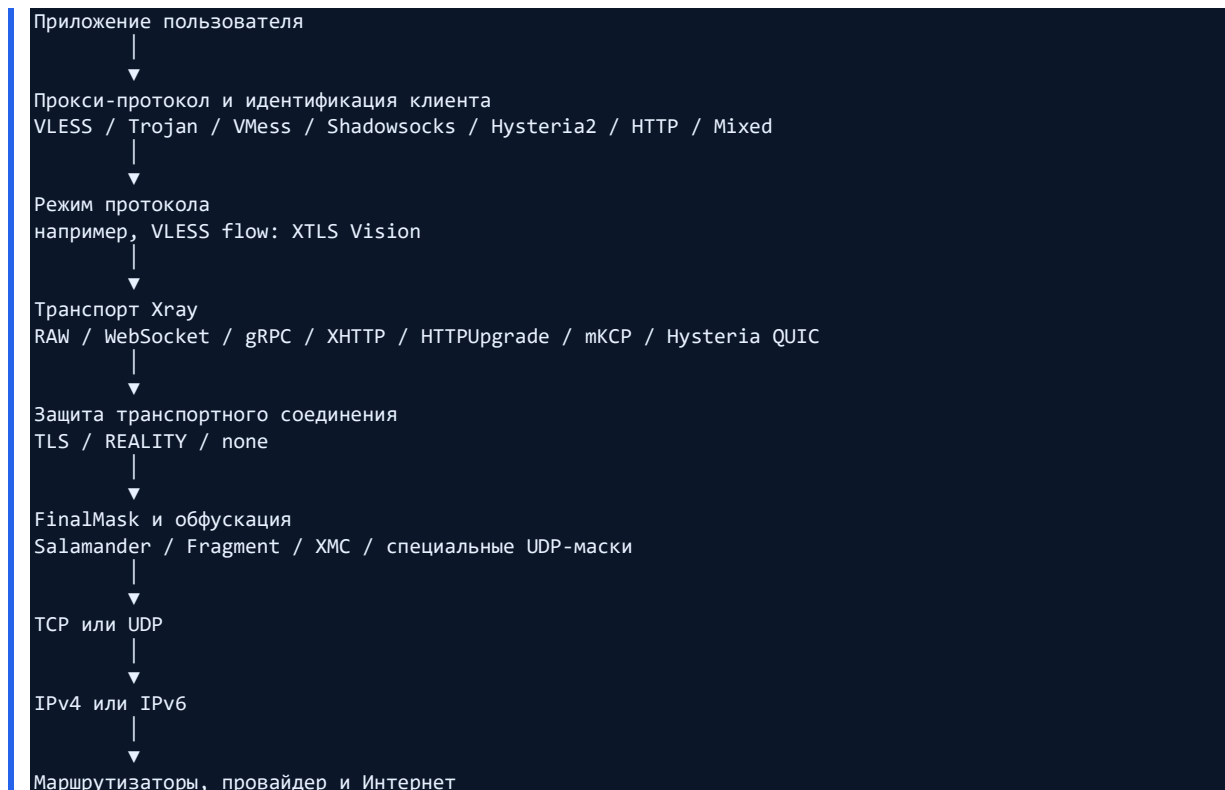
Границы актуальности

- Здесь описаны стабильные и реально используемые возможности. Экспериментальная маска Gecko не включена в рабочие рекомендации.
- TUN сохранён во фронтенде 3x-ui для чтения старых записей, но текущий серверный валидатор не принимает его как новый Xray inbound. Он рассматривается только как совместимость с прежними конфигурациями.
- Поле **External Proxy**, присутствовавшее в старом редакторе inbound, в v3.6.0 заменено страницей **Hosts**. Старые записи продолжают читаться, но редактировать их следует уже через Hosts.
- 3x-ui развивается быстро. Перед применением справочника к другой версии панели нужно проверить changelog и схему этой версии.

Официальные точки сверки: [3x-ui v3.6.0](#), [документация 3x-ui по inbound](#), [структура конфигурации Xray](#).

1. Карта соединения: что находится на каком уровне

Многие ошибки начинаются с попытки сравнить сущности разных уровней. VLESS нельзя корректно сравнить с TLS, а REALITY — с TCP: они отвечают на разные вопросы.



Это практическая модель Xray, а не буквальное переложение OSI. Несколько технологий пересекают границы уровней, но такая схема помогает выбирать совместимые поля.

1.1. IP-уровень

IP отвечает за доставку пакета между адресами. На каждом участке маршрута маршрутизируемый адрес назначения должен быть известен сети. TLS, REALITY, ECH и прокси-протокол не могут сделать IP невидимым для маршрутизаторов.

Можно скрыть **origin IP от конкретного наблюдателя**, поставив перед сервером CDN, relay, reverse проху или cloud edge. Тогда провайдер клиента видит адрес посредника, а посредник знает адрес backend. Подробнее — в главе 11.

1.2. TCP и UDP

TCP создаёт соединение, подтверждает доставку, восстанавливает порядок и регулирует поток. Подходит для RAW, WebSocket, gRPC, XHTTP и HTTPUpgrade.

UDP передаёт отдельные датаграммы без TCP-подтверждений. На нём работают QUIC/Hysteria2 и часть специальных режимов. Потерю, порядок и контроль перегрузки в таком случае реализует верхний протокол, например QUIC.

1.3. Транспорт Xray

Транспорт определяет, как поток прокси-протокола переносится поверх TCP или UDP.

Транспорт в панели	База	Для чего применяют	Практическое замечание
RAW (в JSON исторически tcp)	TCP	Минимум накладных расходов, прямое соединение	Основной вариант для VLESS Vision + REALITY
WebSocket	HTTP/1.1 поверх TCP	Работа через reverse проху/CDN	Удобен и совместим, но имеет HTTP-накладные расходы
gRPC	HTTP/2 поверх TCP	Проксирование через HTTP/2-инфраструктуру	Требует корректных ALPN, service name и поддержки проху
XHTTP	HTTP поверх TCP	Гибкие режимы загрузки/выгрузки, CDN и разделение каналов	Много параметров; значения клиента и сервера должны совпадать
HTTPUpgrade	HTTP/1.1 Upgrade поверх TCP	Схема, похожая на WebSocket, с меньшей спецификой WS	Нужна поддержка Upgrade у промежуточного проху
mKCP	UDP	Собственный надёжный поток поверх UDP	Обычно заметный профиль и расход; TLS/REALITY в текущем UI не предлагаются
Hysteria2	QUIC/UDP	Производительность на потерях и нестабильных линиях	TLS обязателен; UDP может блокироваться или ограничиваться

Актуальная матрица транспорта панели приведена в [документации Зх-ui](#).

1.4. TLS, REALITY и FinalMask — не одно и то же

- **TLS** шифрует транспорт и подтверждает сервер сертификатом.
- **REALITY** модифицирует TLS-рукопожатие Xray и использует реальный сторонний target как правдоподобное прикрытие неавторизованных соединений. Это не отдельный прокси-протокол.
- **FinalMask** меняет внешний вид уже выбранного транспорта. Маска не обязана быть шифрованием и должна одинаково поддерживаться на клиенте и сервере.
- **uTLS fingerprint** изменяет форму ClientHello, имитируя семейство клиентов. Он не превращает Xray в полноценный браузер.

Практическая матрица security текущего редактора:

Security	Протоколы	Транспорты	Примечание
none	Зависит от протокола	Доступные transport без защиты	Только доверенный/внутренний участок; публичный credential и трафик могут быть уязвимы
TLS	VLESS, VMess, Trojan, Shadowsocks; Hysteria2 всегда TLS	RAW, WS, gRPC, XHTTP, HTTPUpgrade; у Hysteria собственный QUIC transport	Сертификат и SNI должны пройти проверку
REALITY	VLESS, Trojan	RAW, XHTTP, gRPC	Требуется совместимый client core, target, serverNames и ключи

UI скрывает несовместимые сочетания. Принудительная сборка другой комбинации через Advanced не означает, что core или клиент её примут.

2. Прокси-протоколы: VLESS, Trojan и Hysteria2

2.1. Краткое сравнение

Свойство	VLESS	Trojan	Hysteria2
Роль	Лёгкий протокол идентификации и переноса Xray	Протокол с парольной аутентификацией, рассчитанный на TLS-подобную схему	Прокси поверх QUIC/UDP
Базовый транспорт	RAW, WS, gRPC, XHTTP, HTTPUpgrade, mKCP	RAW, WS, gRPC, XHTTP, HTTPUpgrade, mKCP	Собственный Hysteria transport поверх QUIC
Защита	none, TLS или REALITY по допустимой комбинации	Обычно TLS; REALITY доступен на совместимых транспортных вариантах	TLS обязателен
Идентификатор клиента	UUID и, при необходимости, VLESS Encryption/Auth	Password	Auth string
Сильная сторона	Максимальная гибкость Xray; Vision/REALITY	Простая модель доступа и широкая совместимость	Потери, высокая задержка, мобильные и нестабильные сети
Ограничение	Много комбинаций и требований совместимости	Нет преимуществ Vision как у VLESS	Зависимость от UDP и более заметный QUIC-профиль

2.2. VLESS

VLESS отделяет аутентификацию от транспортного шифрования. Поле `decryption` на inbound обычно равно `none`: защиту обеспечивает TLS/REALITY либо внешний доверенный контур. Незащищённый публичный VLESS применять не следует.

Ключевые поля:

- `id` — UUID клиента;
- `flow` — режим XTLS; актуальные значения панели: пусто, `xtls-rprx-vision`, `xtls-rprx-vision-udp443`;
- `encryption` — текущий механизм VLESS Encryption/Auth, а не TLS;
- `testpre`, `testseed` — дополнительные параметры проверки/посева для новых режимов VLESS; изменять их стоит только при совпадающей поддержке клиента;
- `reverseTag`, `reverseSniffing` — обратный VLESS-канал и его sniffing, специальный сценарий.

Генератор VLESS Auth в v3.6.0 предлагает семейства `x25519`, `x25519_xorpub`, `x25519_random`, `mlkem768`, `mlkem768_xorpub`, `mlkem768_random`. Это отдельный слой аутентификации VLESS. Он не заменяет TLS/REALITY и требует совместимого клиентского ядра.

Подробно значения `flow`, XTLS Vision и их ограничения разобраны в следующем разделе.

2.3. VLESS Flow и XTLS Vision

Что означает flow

`flow` — параметр VLESS-пользователя и VLESS outbound, выбирающий алгоритм XTLS. Это не транспорт и не transport security:

VLESS	← протокол
flow: xtls-rprx-vision	← алгоритм обработки потока XTLS
RAW или XHTTP	← транспорт
TLS / REALITY / VLESS Enc	← защита подходящей схемы
TCP/IP	← нижние уровни

Flow клиента и сервера должны быть совместимы. Выбор значения только в панели или только в клиенте не создаёт рабочую XTLS-сессию.

Актуальные значения

Значение в 3x-ui	Поведение	Когда выбирать
пусто / none в UI	Обычный VLESS-поток без XTLS Vision; в JSON это пустая строка или отсутствие flow, но не буквальная строка "none"	Совместимость, намеренно обычный TLS proxy или схема, где Vision недоступен
xtls-rprx-vision	XTLS Vision с случайным заполнением внутреннего handshake; клиент перехватывает назначения UDP/443, чтобы приложения по возможности перешли с QUIC на TCP HTTPS	Основной вариант для VLESS + RAW + TLS/REALITY и обычных браузерных сценариев
xtls-rprx-vision-udp443	Тот же Vision, но клиент не перехватывает destination UDP/443	Когда приложение обязательно использует QUIC и не умеет корректно вернуться к TCP

Важно: `udp443` относится к **целевому UDP-порту 443 внутри проксируемого трафика**, а не к порту, на котором слушает VLESS inbound.

В Xray-core v26.7.28 inbound принимает только пустой flow или `xtls-rprx-vision`. Суффикс `-udp443` имеет смысл на клиентском VLESS outbound. Поэтому 3x-ui v3.6.0 работает асимметрично:

```

В базе 3x-ui и клиентской ссылке:
flow = xtls-rprx-vision-udp443

В сгенерированном server inbound Xray:
flow = xtls-rprx-vision

```

Это не потеря настройки. Панель намеренно нормализует серверную запись до базового Vision, а различие «перехватывать ли UDP/443» передаёт клиенту в subscription/share link.

Как работает обычный TLS proxy

При посещении HTTPS-сайта существуют два разных защищённых участка:

```

Внешняя защита proxy:
клиент Xray == TLS/REALITY == сервер Xray

```


Внутренняя прикладная защита:
браузер = TLS сайта = конечный веб-сервер

В обычном TLS proxy внутренние TLS records сайта целиком переносятся внутри внешнего защищённого tunnel. Такой TLS-in-TLS создаёт дополнительные копирования данных, а размеры и последовательности records могут иметь характерный профиль.

Что меняет Vision

XTLS Vision:

1. добавляет случайное заполнение к внутреннему handshake, уменьшая стабильность его размерных признаков;
2. распознаёт подходящий внутренний TLS-поток, не расшифровывая HTTPS-содержимое сайта;
3. после завершения необходимой служебной части старается передавать уже зашифрованные данные внутреннего TLS напрямую, без лишней обработки вторым слоем;
4. на подходящем Linux TCP-пути может передать копирование ядру ОС через `splice`.

Vision не подменяет сертификат конечного сайта, не получает plaintext HTTPS и не отменяет проверку сертификата браузером. Он оптимизирует перенос уже зашифрованного прикладного потока.

Direct copy и Linux splice

`splice` — системная функция Linux, позволяющая ядру передавать TCP-данные между файловыми дескрипторами без постоянного копирования через память процесса Xray. Это уменьшает CPU overhead и переключения контекста.

Для наиболее полного эффекта нужны:

- Linux на узле, где возможен `splice`;
- чистый TCP-путь на соответствующем участке;
- VLESS с Vision;
- TLS 1.3 во внутреннем прикладном соединении для direct-copy сценария;
- отсутствие промежуточного слоя, который обязан заново разбирать/упаковывать каждую порцию данных.

Если условия не выполнены, соединение не обязано ломаться: Xray остаётся на обычной обработке, а выгода будет меньше. На Windows, iOS/Android или при нетипичной транспортной цепочке не следует ожидать Linux `splice`, хотя padding и протокольная логика Vision всё ещё могут использоваться.

При активном [splice](#) статистика скорости/трафика способна обновляться с задержкой и окончательно учитываться после закрытия соединения: userspace Xray не видит каждый переданный ядром фрагмент в реальном времени.

Поддерживаемые сочетания в 3x-ui v3.6.0

Схема	Flow доступен	Что происходит
VLESS + RAW/TCP + TLS	Да	Классический Vision; для внутреннего TLS 1.3 возможен direct copy/splice
VLESS + RAW/TCP + REALITY	Да	Типовая серверная схема: REALITY защищает внешний участок, Vision оптимизирует внутренний TLS-поток
VLESS + XHTTP + VLESS Encryption	Да	Vision «проникает» через VLESS Encryption; при не-TCP нижнем пути полного TCP splice не будет
VLESS + XHTTP без VLESS Encryption	Нет в штатном UI	Нет слоя VLESS Encryption, на который рассчитана эта комбинация
VLESS + WS/gRPC/HTTPUpgrade/mKCP без VLESS Encryption-сценария	Нет в штатном UI	Транспорт требует собственной упаковки, классический RAW Vision не применяется
VMess/Trojan/Shadowsocks	Нет	flow относится к VLESS; совпадение TLS/REALITY само по себе не добавляет Vision

На уровне Xray-core VLESS Encryption снимает ограничение по нижнему транспорту: если прямое TCP-копирование невозможно, Vision экономит обработку слоя VLESS Encryption. Панель v3.6.0 явно открывает этот дополнительный вариант для XHTTP + включённого VLESS Encryption.

REALITY и Vision выполняют разные задачи:

- REALITY отвечает за внешний client↔server handshake и его вид;
- Vision отвечает за обработку потока VLESS и внутреннего TLS;
- uTLS fingerprint формирует ClientHello внешнего соединения.

Поэтому комбинация VLESS + RAW + REALITY + Vision + uTLS логична, но ни один из элементов не скрывает destination IP сервера.

Почему обычный Vision перехватывает UDP/443

Браузер может выбрать HTTP/3 поверх QUIC/UDP вместо HTTPS поверх TCP. UDP-поток нельзя передать через TCP [splice](#), поэтому доля трафика, для которой Vision получает максимальную выгоду, уменьшается.

`xtls-rprx-vision` блокирует/перехватывает проксируемый destination UDP/443, рассчитывая, что приложение повторит соединение по TCP. Он **не конвертирует QUIC в TCP**. Если приложение не умеет fallback, соединение не заработает.

`xtls-rprx-vision-udp443` оставляет QUIC разрешённым. Выбирайте его, когда подтверждено, что конкретное приложение требует UDP/443. При этом:

- inbound/outbound/routing должны действительно переносить UDP;
- Vision splice к этому QUIC-потoku не применяется;
- сетевой профиль и блокируемость UDP остаются свойствами QUIC;
- для большинства браузеров базовый `xtls-rprx-vision` остаётся более предсказуемым вариантом.

Мух, XUDP и XMUX

Зх-ui не включает обычный outbound Мух, если у VLESS задан непустой `flow`. Vision рассчитывает на свойства отдельного потока, тогда как Мух упаковывает несколько логических соединений в одно транспортное соединение.

Не следует смешивать:

- suffix `-udp443` — разрешение клиенту не перехватывать destination QUIC;
- Мух `xudpProxyUDP443` — политика UDP/443 внутри XUDP;
- ХНТТР XMUX — собственное мультиплексирование ХНТТР.

В классической RAW + Vision схеме обычный Мух оставляют выключенным. Для ХНТТР используются параметры XMUX самого ХНТТР, если они нужны и поддерживаются обеими сторонами.

Совместимость клиента и диагностика

Проверьте:

1. клиентское ядро поддерживает выбранный flow и текущие поля REALITY/VLESS Encryption;
2. share link действительно содержит `flow=xtls-rprx-vision` или `flow=xtls-rprx-vision-udp443`;
3. server runtime содержит для пользователя `xtls-rprx-vision`;
4. transport/security/VLESS Encryption входят в разрешённую комбинацию;
5. обычный Мух не включён одновременно с flow;
6. при `udp443` UDP разрешён firewall, routing и клиентом;
7. ошибка не вызвана различием UUID, SNI, short ID, VLESS Encryption keys или версии core.

Если серверному пользователю назначен Vision, а клиент приходит без совместимого flow, Xray отклонит такой режим. Если старый клиент не знает новое значение, решение — обновить ядро или выдать ему отдельный совместимый профиль, а не вручную подменять неизвестные поля.

Устаревшие XTLS-значения

`xtls-rprx-direct`, `xtls-rprx-origin` и `streamSettings.security: "xtls"` относятся к старым поколениям конфигурации. Xray-core v26.7.28 принимает в VLESS flow только пустое значение, `xtls-rprx-vision` и — на outbound — `xtls-rprx-vision-udp443`. Старые значения нельзя копировать в актуальную конфигурацию; для нового подключения используют TLS или REALITY как transport security и Vision как VLESS flow.

Официальная семантика flow, UDP/443 и splice: [VLESS outbound Xray](#) и [VLESS inbound Xray](#).

2.4. Trojan

Trojan использует пароль клиента. На inbound его собственная форма минимальна: список клиентов и, для допустимой RAW-схемы, fallbacks. Большинство важных решений находятся в Stream Settings: транспорт, TLS/REALITY, SNI, ALPN и сертификаты.

Trojan удобно применять, когда нужна простая парольная модель и совместимый клиент. Пароль должен быть случайным и уникальным для пользователя; повторное использование затрудняет отзыв доступа и аудит.

2.5. Hysteria2

Hysteria2 в текущем Xray работает как версия 2 поверх QUIC/UDP. В панели выбирается протокол `hysteria`, но в ссылках и клиентах он обычно представлен как Hysteria2.

Основные элементы:

- клиентский `auth` — секрет доступа;
- `udpIdleTimeout` — сколько секунд неактивная UDP-сессия сохраняется;
- `masquerade` — ответ неавторизованному HTTP-запросу: обычный 404, проху, каталог файлов или статическая строка;
- TLS — обязательный слой с SNI/ALPN/сертификатом;
- FinalMask `Salamander` — парольная UDP-обфускация;
- QUIC-параметры — congestion, BBR profile, brutal up/down, UDP Hop, idle/keepalive, лимиты потоков и окна.

Hysteria2 полезна не потому, что «UDP всегда быстрее», а когда QUIC-контроль перегрузки лучше соответствует линии. В сетях с блокировкой UDP нужен резервный TCP-вариант.

2.6. Другие рабочие inbound-протоколы панели

- **VMess** — исторический протокол экосистемы V2Ray/Xray. Поля клиента: UUID, `security`, `alterId`; для новых конфигураций `alterId` обычно 0.
 - **Shadowsocks** — пароль и cipher. В v3.6.0 доступны `aes-256-gcm`, `chacha20-poly1305`, `chacha20-ietf-poly1305`, `xchacha20-ietf-poly1305` и семейство SS-2022: `2022-blake3-aes-128-gcm`, `2022-blake3-aes-256-gcm`, `2022-blake3-chacha20-poly1305`. У SS-2022 модель master/client passwords отличается от прежних AEAD-вариантов.
 - **WireGuard** — L3-туннель с ключами peer, адресами и AllowedIPs. Это не маскировочный HTTP-протокол.
 - **HTTP** — обычный HTTP проху с учётными записями; без TLS внешний наблюдатель видит содержимое HTTP-прокси-сессии.
 - **Mixed** — совмещённый HTTP/SOCKS-вход, с auth/noauth и опциональным UDP.
 - **Tunnel** — перенаправление адреса/порта и `followRedirect`, аналог сценария dokodemo-door.
 - **MTPROTO** — Telegram проху через управляемый 3x-ui процесс `mtg-multi`, а не Xray inbound. У него отдельные FakeTLS/domain-fronting/IPv4/IPv6 параметры.
-

3. TLS, REALITY, SNI, ECH, ALPN и uTLS

3.1. TLS по шагам

Упрощённый TLS-сеанс выглядит так:

```
ClientHello: SNI, ALPN, версии, наборы шифров, расширения
↓
ServerHello + сертификат
↓
Проверка имени/цепочки или pin
↓
Согласование ключей
↓
Зашифрованные данные транспорта
```

TLS скрывает прикладной трафик, но не маршрутизируемый IP. Обычный ClientHello также раскрывает SNI; ECH может скрыть внутренний ClientHello при выполнении всех условий.

3.2. SNI

SNI (Server Name Indication) — имя сайта, которое клиент передаёт серверу в ClientHello, чтобы один IP мог обслуживать несколько сертификатов и виртуальных хостов.

В Зх-ui поле SNI встречается в нескольких контекстах:

- TLS `serverName` на клиентской стороне — имя, отправляемое серверу;
- TLS `rejectUnknownSni` на сервере — отклонить рукопожатие с неизвестным именем;
- REALITY `serverNames` — разрешённые имена клиента, обычно совпадающие с SAN сертификата target;
- fallback `name` — отбор по принятому SNI;
- External Proxy/Hosts `sni` — SNI следующего участка.

SNI не является паролем. Несовпадение SNI, сертификата, reverse proxy и ALPN часто приводит к ошибке рукопожатия. Подмена SNI не скрывает IP и не гарантирует, что соединение выглядит как настоящий браузер.

3.3. ALPN

ALPN согласует прикладной протокол внутри TLS. Типичные значения:

- `h2` — HTTP/2, необходим для классического gRPC;
- `http/1.1` — WebSocket и HTTPUpgrade;
- `h3` — HTTP/3/QUIC, когда его поддерживает конкретная схема.

ALPN не шифрует трафик. Он должен соответствовать транспорту и промежуточному proxy. Список без необходимости расширять не следует: необычная комбинация становится частью fingerprint.

3.4. uTLS fingerprint

Поле `fingerprint` управляет профилем ClientHello: `chrome`, `firefox`, `safari`, `ios`, `android`, `edge`, `360`, `qq`, `random`, `randomized`, `randomizednoalpn`, `unsafe` или отсутствие имитации.

uTLS имитирует главным образом форму ClientHello. Дальнейшее поведение соединения, порядок HTTP-заголовков, трафик и тайминги всё равно могут отличаться от браузера. Обычно выбирают распространённый стабильный профиль, поддерживаемый клиентом, и не меняют его без причины.

3.5. ECH

Encrypted ClientHello шифрует внутреннюю часть ClientHello, включая реальный SNI. Снаружи остаются IP, транспортные признаки и внешний ClientHello с публичным именем. ECH требует совместимых клиента, сервера, конфигурации ECH и механизма доставки параметров, обычно DNS HTTPS/SVCB.

Поля панели:

- `echServerKeys` — ключи ECH на принимающей стороне;
- `echConfigList` — конфигурация ECH на клиентской стороне;
- генерация/очистка пары ECH;
- `echSockopt` — сетевые параметры отдельного ECH-диалера: `dialerProxy`, `domainStrategy`, `tcpFastOpen`, `tcpMptcp`.

ECH не скрывает IP. Для прикладного sniffing внутренний SNI может стать недоступен: Xray увидит только доступные внешние метаданные. В текущей реализации Xray внешний ALPN формируется отдельно; актуальные детали следует проверять в [документации TLS Xray](#).

3.6. Все актуальные TLS-поля в редакторе

Поле	Назначение	Когда менять
<code>serverName</code>	SNI клиента	Когда соединение идёт по имени, отличному от <code>address</code>
<code>cipherSuites</code>	Разрешённые наборы шифров	Обычно оставить <code>auto</code> ; ручное ограничение ухудшает совместимость
<code>minVersion</code> , <code>maxVersion</code>	Границы TLS, обычно 1.2–1.3	Для требований совместимости/политики
<code>fingerprint</code>	Профиль uTLS ClientHello	Для правдоподобного клиентского отпечатка
<code>alpn</code>	Допустимые прикладные протоколы	Согласовать с WS/gRPC/ХТТТР/HTTPUpgrade
<code>curvePreferences</code>	Предпочтения key exchange: X25519MLKEM768, X25519, P-256/384/521	Оставлять по умолчанию без явной политики

Поле	Назначение	Когда менять
rejectUnknownSni	Отклонять неизвестный SNI	На сервере с известным набором имён
disableSystemRoot	Не использовать системные корневые CA	Только с собственными trust anchors
enableSessionResumption	Разрешить возобновление TLS-сессий	Уменьшает повторную передачу сертификата; это не Xray 0-RTT
verifyPeerCertByName	Имя для проверки сертификата peer	Когда проверяемое имя отличается от sent SNI/address
pinnedPeerCertSha256	SHA-256 pins сертификата peer	Закрытый контур; обновлять при ротации сертификата
masterKeyLog	Запись TLS session secrets	Только краткая диагностика; файл позволяет расшифровать захваченный трафик

Сертификаты TLS

Каждая запись сертификата содержит:

- `useFile` — читать файлы или хранить inline;
- `certificateFile`, `keyFile` — пути к PEM;
- `certificate`, `key` — inline PEM;
- `ocspStapling` — интервал обновления OCSP staple в секундах;
- `oneTimeLoading` — загрузить один раз, без дальнейшего чтения файла;
- `usage` — `encipherment`, `verify` или `issue`;
- `buildChain` — достраивать цепочку при `issue`.

Pinning может заменить обычную проверку CA/имени. Это повышает контроль, но создаёт операционный риск: после ротации сертификата клиенты перестанут подключаться, пока pin не обновлён.

3.7. REALITY

REALITY — транспортная защита Xray, изменяющая TLS-рукопожатие. Она доступна в текущей панели для VLESS/Trojan на RAW, XHTTP и gRPC. Это не CDN и не способ скрыть IP сервера. Точная семантика описана в [официальной документации REALITY](#).

```

Совместимый клиент + правильные параметры
→ REALITY-сессия Xray

Сканер / неверный клиент
→ соединение передаётся REALITY target

```


Поля REALITY

Поле	Назначение
show	Диагностический вывод REALITY; в штатной работе выключен
xver	Версия PROXY protocol к target: 0 — нет, 1 — v1, 2 — v2
fingerprint	uTLS-профиль клиента
target	Реальный TLS-сервис host:port, принимающий неавторизованный трафик; старое имя поля — dest
serverNames	Допустимые SNI; wildcard не используется, имена должны соответствовать target
maxTimediff	Допустимая разница времени клиента и сервера в миллисекундах; 0 отключает проверку
minClientVer, maxClientVer	Диапазон объявляемых версий клиента
shortids	Список идентификаторов: чётное число hex-символов, до 16; возможно пустое значение
privateKey	Серверный X25519 secret
publicKey / password	Клиентская часть пары; новая терминология Xray использует password, UI может показывать прежнее publicKey
spiderX	Начальный путь для spider/crawler-поведения клиента
mlds65Seed	Серверный seed ML-DSA-65
mlds65Verify	Клиентский verify key ML-DSA-65
masterKeyLog	Диагностический TLS key log
limitFallbackUpload, limitFallbackDownload	Ограничение неавторизованного трафика: afterBytes, bytesPerSec, burstBytesPerSec

Пустой `minClientVer` не обязательно означает «без ограничений»: в текущем core применяется встроенный минимум. Искусственно снижать его ради старого стороннего клиента следует только осознанно — это допускает старые TLS-отпечатки.

Для target предпочтителен стабильный TLS-сервис в том же ASN/сетевом окружении, который действительно отвечает на выбранные SNI/ALPN. REALITY пересылает target неавторизованные запросы; неудачный target может выдать конфигурацию, создать нагрузку или быть недоступным из сети сервера. Встроенный scanner панели помогает проверить кандидата.

Ограничители fallback сами формируют наблюдаемый профиль скорости, поэтому это не универсальное улучшение маскировки. Их используют как защиту ресурса, а не «для незаметности».

4. Sniffing: что он видит и зачем нужен

Sniffing выполняется после принятия и расшифровки inbound-трафика. Xray пытается восстановить имя назначения из доступных метаданных: HTTP Host, TLS ClientHello SNI, QUIC или FakeDNS. Затем это имя можно использовать в Routing.

```
Клиент
→ inbound: доступ и расшифровка
→ Sniffing: имя/метаданные
→ Routing: первое совпавшее правило
→ outbound или balancer
→ Интернет
```

Sniffing **не расшифровывает содержимое HTTPS**, не скрывает IP и не является антивирусным DPI.

4.1. Поля Sniffing

Поле	Значение
enabled	Включить анализ
destOverride	Какие источники могут заменить назначение: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
metadataOnly	Использовать только метаданные/FakeDNS, не ожидать payload
routeOnly	Применить распознанное имя только к маршрутизации, не подменять фактическое назначение
ipsExcluded	IP/CIDR, для которых sniffing не применяется
domainsExcluded	Домены, для которых sniffing не применяется

4.2. Выбор режима

- Для доменных правил обычно включают `tls,http,quic` и рассматривают `routeOnly=true` как безопасный старт.
- `routeOnly` важен для приложений, где подмена назначения на sniffed domain ломает соединение; исходный IP сохраняется, а домен участвует только в выборе маршрута.
- `metadataOnly` полезен для FakeDNS и протоколов, где сервер говорит первым. Без него ожидание первых байтов клиента может ломать SMTP-подобные сценарии.
- Исключения используют для push-сервисов, голосовой связи, IoT и других известных конфликтов.
- ECH может скрыть внутренний SNI, поэтому маршрутизация по домену становится неполной. Иногда задачу решает контролируемый DNS/FakeDNS, но отключать ECH приложения без необходимости не следует.

Подробные ограничения `routeOnly` и `metadataOnly` описаны в [InboundObject Xray](#).

5. DPI: базовые принципы и классификация прокси-трафика

DPI (Deep Packet Inspection) — не один конкретный алгоритм, а класс систем, которые сопоставляют пакеты и потоки с правилами или моделью классификации. DPI применяют для сетевой безопасности, QoS, обнаружения атак, корпоративной политики и цензурной фильтрации.

Шифрование ограничивает содержательный анализ, но не делает поток невидимым. Даже TLS 1.3 допускает анализ по длинам и времени зашифрованных пакетов — это прямо отмечено в [RFC 8446](#).

5.1. Где DPI находится в сети

```
Клиент
→ домашний/офисный router
→ сеть доступа провайдера
→ DPI / firewall / traffic classifier
→ магистраль или peering
→ сервер
```

Система может видеть один участок, оба направления потока или агрегированную статистику из нескольких узлов. Возможности различаются: простой firewall проверяет несколько полей пакета, а stateful DPI восстанавливает TCP/QUIC flow, хранит состояние и связывает его с последующими соединениями.

5.2. Какие уровни анализируются

Уровень наблюдения	Примеры признаков
IP и инфраструктура	Destination/source IP, IPv4/IPv6, ASN, hosting-провайдер, reputation, частота смены адреса
TCP/UDP	Порты, flags, размеры окон, MSS, retransmissions, продолжительность, число параллельных потоков
TLS/QUIC handshake	SNI без ECH, ALPN, версии, cipher suites, extensions, порядок/набор параметров, сертификат сервера
Прикладной протокол без шифрования	HTTP Host/path/method, DNS name, сигнатуры VLESS/VMess/MTPROTO и другие поля payload
Статистика потока	Направления, длины пакетов, интервалы, bursts, отношение upload/download, idle periods
Поведение endpoint	Ответ на корректный, ошибочный и специально сконструированный запрос
Корреляция	Совпадение времени и объёма входящего и исходящего потока в разных точках наблюдения

QUIC шифрует handshake, но Initial использует ключи, выводимые из открытых параметров версии и connection ID. Наблюдатель может извлекать часть ClientHello/transport parameters; открыты также UDP/IP metadata и connection IDs. Границы наблюдаемости описаны в [RFC 9000](#), [RFC 9001](#) и [RFC 9312](#).

5.3. Основные методы DPI

1. **Статические сигнатуры.** Поиск фиксированных magic bytes, полей, длин handshake или характерной последовательности сообщений.
2. **Проверка протокольной корректности.** Поток заявляет TLS/HTTP/QUIC, но его дальнейшее поведение не соответствует стандартному клиенту или серверу.
3. **TLS/QUIC fingerprinting.** Сочетание версий, extensions, ALPN, curves, transport parameters и порядка полей сравнивается с известными реализациями.
4. **Статистическая классификация.** Модель использует размеры, направления и timing, даже когда каждый байт payload зашифрован.
5. **IP/ASN/reputation.** Блокируется известный VPS, proxy/CDN range, отдельный port или набор endpoint без доказательства конкретного протокола.
6. **DNS/SNI-корреляция.** Сопоставляются plaintext DNS, SNI, destination IP и последующее соединение.
7. **Active probing.** После пассивного подозрения система сама подключается к endpoint и отправляет корректные/ошибочные варианты handshake, сравнивая ответ.
8. **Replay и mutation probes.** Повтор или небольшое изменение ранее замеченного начала потока выявляет серверную state machine и защиту от replay.
9. **Долговременная корреляция.** Учитываются повторяемость endpoint, расписание, объёмы и связи между потоками одного абонента.

Активное зондирование прокси-серверов и пассивная классификация полностью зашифрованных протоколов подтверждены измерительными исследованиями: [HTTP: A Probe-Resistant Proxy](#) и [How the Great Firewall Detects and Blocks Fully Encrypted Traffic](#).

5.4. Какие действия выполняет DPI

- разрешить поток;
- записать/пометить для последующего наблюдения;
- ограничить скорость или приоритет;
- отбросить отдельные пакеты/направления;
- внедрить TCP RST или искусственный timeout;
- заблокировать destination IP, port, SNI, DNS response или QUIC/UDP целиком;
- перенаправить DNS/HTTP;
- запустить active probe и заблокировать endpoint только после подтверждения.

Поэтому «не подключается» не всегда означает полную блокировку протокола: это может быть throttling, выборочная потеря UDP, DNS-подмена, IPv4-only правило или временная reputation-блокировка.

5.5. Как DPI классифицирует VLESS и другие протоколы

Схема	Что скрыто	Что остаётся наблюдаемым	Типичные методы DPI
VLESS без TLS/REALITY/VLESS Encryption	Практически ничего на транспортном уровне	VLESS handshake/payload, IP, port, timing	Прямая сигнатура и protocol validation; публично применять нельзя
VLESS + TLS	VLESS payload и трафик внутри TLS	IP, SNI без ECH, ALPN, TLS fingerprint, сертификат, flow statistics	TLS fingerprint, SNI/IP rules, активное подключение, статистика TLS-in-TLS
VLESS + REALITY	VLESS payload; корректный клиент отличается секретными параметрами	IP/ASN, внешний ClientHello, target consistency, packet sizes/timing	Reputation, несоответствие fingerprint/target/ASN, traffic analysis, специально сформированные probes
VLESS + Vision	Внутренний VLESS и TLS payload; уменьшены признаки TLS-in-TLS	Внешний IP/handshake, направления/время/размеры, факт перехвата или разрешения UDP/443	Flow statistics, endpoint reputation, active probing; Vision не является невидимостью
VLESS + WS/gRPC/XHTTP через CDN	Origin может быть скрыт от ISP клиента, VLESS payload защищён TLS	Edge IP, внешний SNI/ALPN, HTTP-поведение; CDN видит свой участок и origin	Проверка HTTP semantics/fingerprint, правила CDN, SNI/IP, объёмно-временная классификация
Trojan + TLS/REALITY	Password и proxy payload	Те же внешние TLS/IP/flow признаки	TLS fingerprint, поведение на неверный password/probe, reputation/statistics
Hysteria2	Payload внутри QUIC/TLS; с Salamander также скрыта стандартная QUIC-сигнатура	UDP/IP/port, размеры, интервалы, connection behavior, высокая длительная UDP-нагрузка	QUIC fingerprint либо generic encrypted-UDP classification; UDP block/throttle; active probes
Shadowsocks/VMess без внешнего TLS-прикрытия	Payload зашифрован	Поток «высокой энтропии» без стандартного TLS/QUIC handshake, длины и ответы на probes	Fully-encrypted-traffic classifier, replay/mutation, active probing, IP reputation
WireGuard	Tunnel payload	Характерный UDP handshake, размеры и keepalive/timing	Protocol signature, UDP/port rules и endpoint reputation
MTPROTO/FakeTLS	Telegram payload	FakeTLS consistency, handshake/packet profile, endpoint	Protocol validation, TLS-semantic mismatch, IP/ASN rules и active probe

Фраза «DPI видит VLESS» обычно неточна. При корректном TLS/REALITY DPI не читает VLESS-команды как открытый текст; он **классифицирует весь внешний поток** по доступным признакам и вероятности.

5.6. SNI, ECH и DNS с точки зрения DPI

- обычный SNI — явный доменный признак в ClientHello;
- ECH скрывает внутренний ClientHello/SNI, но оставляет IP, внешний ClientHello и принадлежность service provider;
- ECH не помогает, если домен раскрыт plaintext DNS или имеет отдельный уникальный IP;
- encrypted DNS скрывает запрос от локального наблюдателя, но resolver и destination IP остаются частями модели доверия;
- сеть может применять политику к самому факту ECH/DoH/QUIC, хотя это повышает число false positives.

Ограничения ECH прямо сформулированы в [RFC 9849](#): ECH сам по себе недостаточен, если имя раскрывается через DNS или видимый адрес сервера.

5.7. Почему нет гарантированно «невидимой» конфигурации

- любое соединение имеет видимый адрес следующего hop;
- маскировка одного признака оставляет другие: timing, длины, IP, handshake и поведение endpoint;
- слишком необычная обфускация сама может стать сигнатурой;
- allowlist-модель требует выглядеть как разрешённый протокол целиком, а не только первые пакеты;
- методы DPI и протоколы меняются, поэтому вчерашний профиль не гарантирует результат сегодня;
- агрессивная классификация даёт false positives, поэтому оператор DPI балансирует точность, стоимость и побочный ущерб.

TLS, REALITY, Vision, CDN и FinalMask уменьшают разные классы признаков, но не обеспечивают абсолютную неразличимость. Правильная формулировка цели: **сделать классификацию менее надёжной и отделить origin от конкретного наблюдателя**, сохраняя приемлемую совместимость.

5.8. Диагностика без догадок

1. Сравнить IPv4 и IPv6.
2. Проверить DNS отдельно от TCP/UDP-connect.
3. Сравнить TCP transport и Hysteria2/UDP.
4. Проверить тот же endpoint из другой сети.
5. Разделить TLS/REALITY handshake error, timeout, reset и throttling.
6. Проверить server logs: дошёл ли запрос до inbound.

7. Сравнить прямой VPS, relay и CDN edge только в разрешённой тестовой среде.
 8. Не делать вывод о DPI по одному неуспешному приложению: причиной могут быть MTU, firewall, NAT, certificate/SNI, ALPN или несовместимое client core.
-

6. Создание inbound: полный каталог полей 3x-ui v3.6.0

Inbound — точка, где Xray слушает адрес/порт, принимает клиента и передаёт запрос в Routing. Не все поля показываются одновременно: форма динамически зависит от protocol, transport и security.

6.1. Общие поля inbound

Поле панели	Назначение	Замечание
Enable	Включить listener	Отключённая запись хранится, но не обслуживает подключения
Remark	Человекочитаемое имя	Не участвует в протоколе; используйте единый шаблон имён
Deploy to / nodeId	Узел, где разворачивать inbound	Доступно для поддерживаемых протоколов в multi-node
Protocol	Тип входа	От него зависят клиенты и Stream Settings
Listen IP	Локальный адрес bind	Пусто/0.0.0.0 — все IPv4-интерфейсы; :: — IPv6 с учётом v6only
Share address strategy	Какой адрес публиковать: node/listen/custom	Bind-адрес и адрес в подписке — разные понятия
Share address	Публикуемый custom hostname/IP	Нужен при NAT, reverse proxy, CDN, DNS-имени
Subscription sort index	Порядок в подписке	Организационное поле
Port	Порт listener	Должен быть свободен и разрешён firewall/security group
Total traffic	Общая квота, отображается в GB	0 — без лимита
Traffic reset	Never/hourly/daily/weekly/monthly	Период автоматического сброса счётчика
Traffic reset day	День месяца 1–31	Только для monthly; учитывайте короткие месяцы
Expiry time	Дата отключения inbound	Отличается от expiry конкретного клиента
Tag	Внутренний идентификатор Xray	Панель обычно формирует его сама; Routing ссылается именно на tag

up, down, время последнего сброса и другие счётчики — служебные поля базы, а не параметры соединения.

6.2. Общие поля клиента

Поле	Назначение
Email / Name	Уникальное имя клиента внутри inbound; используется статистикой и routing user
Enable	Разрешить клиенту доступ
Credential	UUID, password, auth или ключ — зависит от протокола
IP Limit	Максимум одновременно наблюдаемых IP; в типовой установке реализуется связкой логов и Fail2ban
Total GB	Квота клиента; 0 — без лимита
Expiry time	Срок действия клиента
Reset	Период сброса клиентской квоты
Telegram ID	Получатель уведомлений панели
Subscription ID	Стабильный идентификатор индивидуальной подписки
Comment	Административная заметка
Group	Организация клиентов в панели
External links	Дополнительные ссылки/метаданные для пользователя

Для VLESS/VMess credential — UUID, для Trojan — password, для Hysteria — auth, для WireGuard — peer key material. Секреты должны быть уникальными; удаление клиента важнее простого удаления ссылки из устройства.

6.3. Поля по протоколам

VLESS

- `decryption` — как правило `none`;
- `encryption` — строка VLESS Encryption/Auth; `none` при обычной UUID-модели;
- **Generate Auth** — генератор X25519/ML-KEM вариантов;
- `testpre` — дополнительный тестовый prefix-параметр нового VLESS;
- `testseed` — четыре положительных целых; UI предлагает базовый набор `900`, `500`, `900`, `256` в совместимых TCP + TLS/REALITY Vision-сценариях;
- Clients: UUID, flow и общие лимиты;
- Fallbacks — только в допустимой RAW + TLS/REALITY конфигурации.

Trojan

- Clients: password и общие лимиты;
- Fallbacks — при совместимой RAW + TLS/REALITY схеме;
- отдельного поля шифра нет: защита определяется transport security.

Hysteria2

- `version` — фиксированное 2;
- Clients: auth и общие лимиты;
- `udpIdleTimeout` — 2–600 секунд, типично 60;
- `masquerade.type`:
 - пусто — стандартный 404;
 - `proxy` — `url`, `rewriteHost`, `insecure` для внутреннего проху;
 - `file` — каталог `dir`;
 - `string` — `statusCode`, `content`, таблица `headers`;
- TLS обязателен;
- FinalMask обычно `Salamander`; QUIC-поля вынесены в раздел 6.9.

`Masquerade` отвечает не за авторизованный прокси-трафик, а за правдоподобный ответ посетителю без Hysteria auth. Не публикуйте произвольный локальный каталог через тип `file`.

VMess

- Clients: `id`, `security`, `alterId`, общие лимиты;
- `security=auto` обычно оставляют клиентскому ядру;
- `alterId=0` — нормальный современный вариант; ненулевой `alterId` — наследие старых конфигураций.

Shadowsocks

- `method` — cipher из перечня главы 2;
- `password` — inbound/master password; его роль зависит от старого AEAD или SS-2022;
- `network` — TCP, UDP или оба;
- `ivCheck` — проверка повторного IV для подходящих cipher;
- Clients — multi-user записи SS-2022 с методом/паролем и общими лимитами.

Не смешивайте структуру ключей SS-2022 с привычным одиночным паролем старых методов: link и сервер должны сформировать ключи одинаково.

WireGuard

- `secretKey` — приватный ключ интерфейса;
- `publicKey` — вычисляемый UI публичный ключ;
- `mtu` — MTU туннеля;
- `dns` — DNS-адреса для клиентской конфигурации;
- `noKernelTun` — запрет kernel TUN, переход к userspace;
- `domainStrategy` — `ForceIP`, `ForceIPv4`, `ForceIPv4v6`, `ForceIPv6`, `ForceIPv6v4`;
- `peer/client`: private/public/pre-shared key, addresses, `allowedIPs`, `keepAlive` и общие лимиты.

Private key не должен попадать в документацию или скриншоты. `AllowedIPs` одновременно задаёт допустимую сеть реер и выбор маршрута в WireGuard.

HTTP

- Accounts: `user`, `pass`;
- `allowTransparent` — разрешить прозрачный режим;
- внешнюю публикацию без TLS не использовать: basic credentials и запросы не получают транспортной защиты.

Mixed

- Accounts: `user`, `pass`;
- `auth` — password/noauth;
- `udp` — принимать UDP через SOCKS;
- `ip` — IP для UDP-ассоциации.

Mixed заменяет отдельный SOCKS inbound в текущем интерфейсе и одновременно обслуживает HTTP proxy.

Tunnel

- `rewriteAddress`, `rewritePort` — целевое назначение;
- `allowedNetwork` — TCP, UDP или оба;
- `portMap` — таблица отображения портов;
- `followRedirect` — принять исходное назначение прозрачного redirect/TProxy;
- из Stream Settings здесь практически важен Sockopt/TProxy, а не HTTP-транспорт.

MTPROTO / mtg-multi

- `fakeTlsDomain` — домен FakeTLS;

- `domainFronting.ip`, `.port`, `.proxyProtocol` — внешний front;
- `proxyProtocolListener` — принимать PROXY protocol;
- `preferIp` — prefer/only IPv4 или IPv6;
- `debug` — расширенный журнал;
- `throttleMaxConnections` — ограничение соединений;
- `routeThroughXray` — передать трафик в Xray outbound;
- `outboundTag` — какой outbound использовать;
- `publicIpv4`, `publicIpv6` — публикуемые адреса;
- client: FakeTLS secret, `adTag`, общие лимиты.

Это настройки `mtg-multi`. Они входят в возможности Зх-ui, но не являются полями Xray-core.

TUN — только совместимость

Старые записи могут содержать `name`, `mtu`, `gateway`, `dns`, `userLevel`, `autoSystemRoutingTable`, `autoOutboundsInterface`. В v3.6.0 frontend умеет их показать, но backend не принимает создание нового TUN inbound как рабочую Xray-конфигурацию. Для системного TUN используйте поддерживаемый клиент/ядро, а не этот legacy-пункт панели.

6.4. Fallbacks

Fallback позволяет одной RAW TLS/REALITY точке передавать неподходящий трафик другому локальному сервису или дочернему inbound. Правила проверяются по порядку.

Поле	Назначение
Child inbound	Выбрать другой inbound как destination
name / SNI	Совпадение принятого server name
alpn	Совпадение согласованного ALPN
path	Совпадение первого HTTP path
dest	Адрес/порт назначения fallback
xver	PROXY protocol до destination: 0/1/2

`xver` включают только если destination действительно понимает соответствующий PROXY protocol. Иначе его первые байты будут восприняты как испорченный прикладной запрос.

6.5. RAW transport

RAW — минимальный TCP-транспорт.

- `acceptProxyProtocol` — ожидать PROXY v1/v2 перед входным потоком;
- HTTP camouflage request: `version`, `method`, `path`, `headers`;
- HTTP camouflage response: `version`, `status`, `reason`, `headers`.

HTTP camouflage RAW — не полноценный reverse proxy и не шифрование. Если upstream не посылает PROXY protocol, `acceptProxyProtocol` выключают.

6.6. WebSocket, gRPC и HTTPUpgrade

WebSocket

- `acceptProxyProtocol` — принять PROXY protocol до HTTP;
- `host` — ожидаемый Host;
- `path` — WebSocket path;
- `heartbeatPeriod` — период heartbeat в секундах, `0` выключает;
- `headers` — дополнительные HTTP-заголовки.

gRPC

- `serviceName` — имя gRPC service/path;
- `authority` — HTTP/2 `:authority`;
- `multiMode` — multi-mode Xray.

HTTPUpgrade

- `acceptProxyProtocol`;
- `host`;
- `path`;
- `headers`.

Host/path/serviceName должны совпасть на клиенте, reverse proxy и сервере. За TLS-терминатором внутренний Xray inbound может работать без TLS только в доверенном локальном сегменте.

6.7. ХНТТР

ХНТТР предоставляет разные режимы отправки uplink/downlink по HTTP. Большинство расширенных полей стоит оставлять по умолчанию: нестандартная раскладка обязана совпадать у клиента и сервера.

Базовые поля ХНТТР

Поле	Назначение
path	HTTP path
host	Host/authority
mode	auto, packet-up, stream-up, stream-one
headers	Дополнительные HTTP headers
xPaddingBytes	Диапазон padding, UI по умолчанию 100-1000
noSSEHeader	Не использовать SSE-заголовок
noGRPCHeader	Не использовать gRPC-подобный заголовок
uplinkHTTPMethod	Метод HTTP uplink
scMaxEachPostBytes	Максимум данных одного POST
scMinPostsIntervalMs	Минимальный интервал POST клиента
uplinkChunkSize	Размер uplink chunk
scMaxBufferedPosts	Число буферизуемых POST, типично 30
scStreamUpServerSecs	Длительность server stream-up, типичный диапазон 20–80 с
serverMaxHeaderBytes	Максимальный размер заголовков на сервере

Размещение и ключи полей

- xPaddingObfsMode, xPaddingKey, xPaddingHeader, xPaddingPlacement, xPaddingMethod — как формировать/передавать padding;
- sessionIDPlacement, sessionIDKey, sessionIDTable, sessionIDLength — где и как кодировать session ID;
- seqPlacement, seqKey — размещение sequence number;
- uplinkDataPlacement, uplinkDataKey — где переносить uplink data.

Значения Placement определяют, попадает элемент в path/query/header/body. Key задаёт имя соответствующего параметра/заголовка. Table — алфавит генерации ID. Пустые поля позволяют Xray использовать согласованные defaults.

ХМУХ внутри ХНТТР

- Enable XMUX;
- maxConcurrency — число конкурентных потоков на соединение;

- `maxConnections` — число соединений; используется как альтернатива `maxConcurrency`, свежий default панели — 3;
- `cMaxReuseTimes` — максимальное число повторных использований соединения;
- `hMaxRequestTimes` — максимум запросов на HTTP-соединение, типично 600–900;
- `hMaxReusableSecs` — срок повторного использования, типично 1800–3000 секунд;
- `hKeepAlivePeriod` — период keepalive.

`maxConcurrency` и `maxConnections` — разные стратегии ограничения и в UI взаимоисключаются.

Увеличение значений не гарантирует скорость: оно меняет число наблюдаемых соединений, нагрузку и head-of-line поведение.

6.8. mKCP

Поле	Назначение
<code>mtu</code>	Размер пакета, 576–1460; default 1350
<code>tti</code>	Интервал обработки, 10–100 мс; default 20
<code>uplinkCapacity</code>	Оценка uplink capacity
<code>downlinkCapacity</code>	Оценка downlink capacity
<code>cwndMultiplier</code>	Множитель congestion window
<code>maxSendingWindow</code>	Максимальное send window

Старые `header/seed` представлены в новых схемах через FinalMask `mkcp-legacy`. Неверная MTU вызывает фрагментацию или black-hole; сначала проверяют path MTU, затем меняют окна/ёмкость.

6.9. Hysteria/QUIC transport

Поле	Назначение
<code>auth</code>	Транспортный auth клиента
<code>udpidleTimeout</code>	Таймаут неактивной UDP-сессии
<code>congestion</code>	reno, bbr, brutal, force-brutal
<code>bbrProfile</code>	conservative, standard, aggressive
<code>brutalUp, brutalDown</code>	Заявленная скорость для Brutal
<code>udpHop.ports</code>	Диапазон/список портов, например 20000-50000
<code>udpHop.interval</code>	Интервал смены порта, обычно 5–10 с

Поле	Назначение
maxIdleTimeout	QUIC idle timeout, 4–120 с
keepAlivePeriod	QUIC keepalive, 2–60 с
disablePathMTUDiscovery	Не использовать PMTU discovery
maxIncomingStreams	Лимит входящих streams, минимум 8
initialStreamReceiveWindow, maxStreamReceiveWindow	Начальное/максимальное окно stream
initialConnectionReceiveWindow, maxConnectionReceiveWindow	Начальное/максимальное окно connection
debug	Расширенная диагностика QUIC

UDP Нор имеет смысл только когда весь диапазон открыт и направлен на один экземпляр, а клиент поддерживает те же параметры. Нельзя распределять нор-порты случайно между несогласованными backend: QUIC connection ID и состояние должны сохраняться.

6.10. Sockopt

Sockopt управляет системным сокетом, а не прокси-протоколом.

Поле inbound	Назначение
Real client IP preset	Off / Cloudflare headers / PROXY protocol
mark	Linux fwmark для policy routing
tcpKeepAliveInterval	Интервал keepalive probes
tcpKeepAliveIdle	Простой перед началом keepalive
tcpMaxSeg	TCP maximum segment size
tcpUserTimeout	Максимум ожидания неподтверждённых данных
tcpWindowClamp	Ограничение receive window
acceptProxyProtocol	Принимать PROXY protocol на соquete
tcpFastOpen	TCP Fast Open, если поддержан ОС и сетью
penetrate	Специальный режим проникновения/обхода сетевых ограничений Xray; не универсальная оптимизация
v6only	На IPv6 listener запретить IPv4-mapped подключения
tcpcongestion	bbr, cubic, reno, если алгоритм доступен ядру ОС
tproxy	off, redirect, tproxy для прозрачного proxy

Поле inbound	Назначение
trustedXForwardedFor	Доверенные headers: CF-Connecting-IP, X-Real-IP, True-Client-IP, X-Client-IP
customSockopt	Список низкоуровневых socket options

Headers реального IP принимают только от доверенного reverse proxy, а origin firewall закрывают от прямого доступа. Иначе клиент сам подставит `X-Real-IP`. PROXY protocol включают на обеих сторонах одного участка.

В outbound Sockopt дополнительно доступны:

- `dialerProxy` — отправить dial через другой outbound tag;
- `domainStrategy`, `addressPortStrategy` — разрешение адреса/выбор семейства;
- `interface` — исходящий интерфейс;
- `tcpMptcp` — Multipath TCP при поддержке ОС/peer;
- Happy Eyeballs: `tryDelayMs`, `prioritizeIPv6`, `interleave`, `maxConcurrentTry`;
- остальные TCP/mark/TProxy/custom options из таблицы.

6.11. FinalMask и TCP/UDP masks

FinalMask применяется после выбора транспорта/security. Он не заменяет TLS и работает только с клиентом, который реализует ту же маску.

TCP masks

- **fragment**: `packets` (например, TLS hello или диапазон), списки `lengths`, `delays`, `maxSplit`. Дробит начальные данные; чрезмерная фрагментация снижает скорость и сама становится fingerprint.
- **sudoku**: `password`, `ascii`, `customTable`, `paddingMin`, `paddingMax`. Кодирует поток согласованной таблицей/паролем.
- **header-custom**: наборы client/server items. Item может быть `array`, `str`, `hex`, `base64`, иметь `delay`, `rand`/`randRange` или packet. Это конструктор специализированного двустороннего заголовка.
- **XMC**: `hostname`, профили Minecraft session (`username`, UUID, textures value/signature), `password`. Применять только в поддерживаемой XMC-схеме; неполный профиль текущий core отвергает.

UDP masks

- **Salamander**: `password`. Основная стабильная маска для Hysteria2 и поддерживаемых WireGuard-сценариев.

- **mkcp-legacy**: прежние варианты `original`, `aes-128-gcm`, `dns`, `dtls`, `srtp`, `utp`, `wechat`, `wireguard` и их value.
- **header-custom**: отдельные client/server элементы пользовательского заголовка.
- **XDNS**: список доменов для DNS-подобного оформления.
- **XICMP**: datagram mode и список IP для ICMP-подобной схемы.
- **noise**: `reset` и последовательность noise items с packet/random/delay.
- **realm**: `url`, STUN servers и опциональные TLS `serverName`, ALPN, fingerprint и локальный `allowInsecure` этой маски.
- **sudoku** присутствует в схеме core, но в обычном списке UI v3.6.0 не предлагается; рассматривать его следует как advanced JSON, а не типовой выбор.

`allowInsecure` внутри специализированного Realm не следует переносить на общую TLS-настройку: в актуальной форме TLS Зх-ui для проверки используются корректное имя/CA, `verifyPeerCertByName` и pins.

6.12. External Proxy → Hosts

В пользовательских скриншотах старой версии встречались:

- force TLS: same/TLS/none;
- destination, port, remark;
- SNI, fingerprint, ALPN;
- pinned peer certificate SHA-256;
- verify peer certificate by name;
- ECH config list;
- VLESS route.

В v3.6.0 эти поля больше не редактируются как вкладка inbound. Их функциональность перенесена на страницу **Hosts**, а прежние объекты читаются для обратной совместимости. Поэтому в новом справочнике они сохранены как карта миграции, но не как инструкция создавать legacy External Proxy.

7. Xray Configs: что это за раздел

Зх-ui — панель управления, база пользователей и генератор конфигурации. Xray-core — процесс обработки трафика.

```
Зх-ui: UI + база + шаблон
    |
    v
Итоговый JSON Xray
    |
    v
Xray-core: inbounds → sniffing → routing/DNS → outbounds/balancers
```

Раздел **Xray Configs** редактирует глобальный шаблон: Basics, Routing Rules, Outbounds, Balancers, DNS и Advanced. После изменения панель должна собрать валидный JSON и перезапустить/обновить core. Неверная ссылка на tag или синтаксическая ошибка Advanced может остановить все inbounds.

7.1. Basics

General

Поле	Назначение
Freedom Strategy	DNS-стратегия основного direct/freedom: AsIs, UseIP, UseIPv4, UseIPv6, UseIPv6v4, UseIPv4v6, ForceIP, ForceIPv6v4, ForceIPv6, ForceIPv4v6, ForceIPv4
Freedom Happy Eyeballs	Параллельно/с задержкой пробовать IPv4 и IPv6
tryDelayMs	Задержка перед альтернативным семейством
prioritizeIPv6	Предпочитать IPv6 при гонке адресов
Routing Strategy	AsIs, IPfNonMatch, IPOnDemand
Outbound Test URL	URL, которым панель/observer проверяет реальный выход

AsIs сохраняет домен. **IPfNonMatch** сначала проверяет доменные правила и только затем, если совпадения нет, разрешает IP для IP-правил. **IPOnDemand** может запрашивать IP раньше, как только встречается потенциально применимое IP-условие. Выбор влияет и на утечки DNS, и на задержку.

Statistics

- **statsInboundUplink** — считать upload по inbound/user;
- **statsInboundDownlink** — считать download;
- **statsOutboundUplink**, **statsOutboundDownlink** — считать outbound traffic;
- **metrics.listen** — адрес HTTP metrics endpoint;
- **metrics.tag** — внутренний tag metrics/stats listener.

Статистика нужна квотам и отчётам, но добавляет память/CPU. Metrics listener нельзя бездумно публиковать в Интернет: он раскрывает служебные данные.

Connection limits / Policy level 0

- `connIdle` — секунд без трафика до закрытия соединения;
- `bufferSize` — размер буфера в KiB для policy level 0.

Слишком низкий `connIdle` ломает долгие idle-сессии и push; слишком высокий удерживает ресурсы. Большой buffer не всегда ускоряет поток и увеличивает память на соединение.

Logging

Поле	Назначение
Log level	<code>none</code> , <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>
Access log	<code>none</code> или путь, типично <code>./access.log</code>
Error log	пусто/ <code>stderr</code> , <code>none</code> или путь
Mask address	пусто, <code>quarter</code> , <code>half</code> , <code>full</code> — маскировать адреса в логах
DNS log	Записывать DNS-запросы
Reset	Вернуть базовый шаблон панели

`debug` и DNS log включают на ограниченное время: они быстро растят файл и собирают чувствительные домены/IP. Настройте rotation отдельно на уровне ОС.

7.2. Routing Rules

Routing рассматривает правила сверху вниз и применяет **первое совпавшее**. Если ничего не совпало, используется первый/default outbound. Подробная модель приведена в [официальном разделе Routing](#).

Быстрые настройки

- Default outbound — основной выход;
- Block BitTorrent;
- Block IPs / Block domains;
- Direct IPs / Direct domains;
- IPv4 routing domains — домены, которые принудительно разрешаются/выводятся через IPv4.

Это генераторы обычных rules, а не независимый firewall. После применения проверяйте фактический порядок.

Поля визуального редактора rule

Поле	Назначение
Enable	Включить правило
Source IP	IP/CIDR клиента до inbound
Source port	Исходный порт клиента
VLESS Route	Сопоставление специального VLESS route
Network	TCP, UDP или оба
Protocol	Распознанный http, tls, bittorrent, quic
Attributes	Пары key/value распознанных атрибутов
IP	Destination IP/CIDR/geoip:
Domain	full:, domain:, regexp:, geosite: и обычные доменные шаблоны
User	Email/идентификатор Xray user
Port	Destination port/range
Inbound tag	От какого inbound пришёл поток
Outbound tag	Отправить в конкретный outbound
Balancer tag	Передать выбор группе balancer

У правила выбирают либо `outboundTag`, либо `balancerTag`. Одновременное заполнение создаёт неоднозначную/невалидную конфигурацию.

Advanced JSON также поддерживает `localPort`, `localIP`, `process`, `ruleTag` и `webhook` (`url`, `deduplication`, `headers`). Визуальная форма сохраняет такие поля, но не предоставляет им полноценного редактора; менять их нужно только после сверки со схемой текущего core.

Практический порядок правил

1. Служебные API/metrics rules панели
2. Явные deny/block
3. Исключения direct
4. Специальные proxy/balancer rules
5. Общие региональные правила
6. Default outbound

Более узкое правило ставят выше широкого. `geosite:` классифицирует домены, `geoip:` — уже IP; для доменной маршрутизации нужен домен от клиента, DNS или Sniffing.

7.3. Outbounds

Outbound — способ, которым Xray устанавливает следующий участок соединения. Первый outbound шаблона обычно является default, если rule не выбрало другой.

Общие поля outbound

Поле	Назначение
Protocol	freedom, blackhole, dns, vmess, vless, trojan, shadowsocks, wireguard, hysteria, socks, http, loopback
Tag	Уникальное имя для rules, balancers и dialerProxy
Send Through	Локальный source IP для исходящего socket
Target Strategy	AsIs или стратегия разрешения IPv4/IPv6
Stream Settings	RAW/WS/gRPC/xHTTP/HTTPUpgrade, TLS/REALITY, FinalMask и Sockopt
Flow	Протокольный flow, прежде всего VLESS Vision
Mux	Мультиплексирование потоков

Tag должен быть уникальным и стабильным. Его переименование требует обновить rules, selectors, fallbackTag, dialerProxy и subscription references.

Freedom / direct

- `domainStrategy` — как разрешать destination;
- `redirect` — переписать адрес/порт назначения;
- `userLevel` — policy level;
- `proxyProtocol` — отправлять PROXY v1/v2 следующему сервису;
- `fragment: packets, length, interval, maxSplit`;
- `noises`: тип/packet, delay, `applyTo`;
- `finalRules: action, network, port, ip, blockDelay`.

Freedom — прямой выход сервера. Fragment/noise — специальные техники на участке server→destination; они не меняют IP выхода и могут ухудшить совместимость.

Blackhole

- response type: пусто/`none` или `http`;
- используется для блокировки rule;
- HTTP response уместен только когда клиент ожидает HTTP, иначе достаточно тихого закрытия/none.

DNS outbound

- `rewriteNetwork` — UDP/TCP;
- `rewriteAddress`, `rewritePort` — куда переписать DNS-запрос;
- `userLevel`;
- rules: `action` (`direct`, `drop`, `return`, `hijack`), `qType`, `domain`, `rCode`.

Это outbound для перехвата/передачи DNS-трафика, не глобальный DNS-модуль из главы 9.

VLESS outbound

- `address`, `port`, `id`;
- `encryption`, `flow`;
- `reverseTag`, `reverseSniffing`;
- `testpre`, `testseed`;
- Stream Security: none/TLS/REALITY в допустимой комбинации.

VMess outbound

- `address`, `port`, `id`, `security`;
- `transport/security/mux`;
- новый outbound должен совпадать с удалённым inbound по UUID и stream fields.

Trojan outbound

- `address`, `port`, `password`;
- `transport/security`;
- обычно требуется TLS или REALITY-совместимая схема.

Shadowsocks outbound

- `address`, `port`, `password`, `method`;
- `uot` — UDP over TCP;
- `UoTVersion` — версия 1 или 2.

УоТ используют лишь при совпадающей поддержке сервера и клиента; он переносит UDP-семантику через TCP со собственными TCP задержками.

WireGuard outbound

- `secretKey`, вычисляемый `pubKey`;
- `address`, `mtu`, `domainStrategy`, `reserved`, `noKernelTun`;
- peers: `publicKey`, `preSharedKey`, `allowedIPs`, `endpoint`, `keepAlive`.

WireGuard outbound меняет egress IP на адрес выхода WireGuard peer, если AllowedIPs/rules охватывают destination. Он не скрывает IP inbound сервера от клиента.

Hysteria outbound

- `address`, `port`, `version=2`;
- stream Hysteria: auth, congestion/QUIC settings;
- TLS и совместимый FinalMask.

SOCKS и HTTP outbound

- SOCKS: `address`, `port`, опциональные `user`, `pass`;
- HTTP: `address`, `port`, `user`, `pass`, `headers`.

Это следующий проху-хор. Проверяйте, где заканчивается TLS и кто видит конечное назначение.

Loopback

- `inboundTag` — повторно передать поток конкретному inbound;
- `sniffing` — та же схема `enabled/destOverride/metadataOnly/routeOnly/exclusions`.

Loopback — служебный механизм повторного ввода. Зх-ui использует скрытые loopback tags вида `_bl_...` для fallback одного balancer на другой. Самостоятельно строить loopback-циклы нельзя.

Mux

- `enabled`;
- `concurrency` — логические потоки на mux connection;
- `xudpConcurrency` — конкурентность XUDP;
- `xudpProxyUDP443` — `reject`, `allow`, `skip`.

Mux уменьшает число transport connections, но один перегруженный/потерянный канал может повлиять на несколько потоков. Для Vision и некоторых transport комбинаций UI запрещает Mux.

Outbound subscriptions

Подписка импортирует набор поддерживаемых share links в Outbounds:

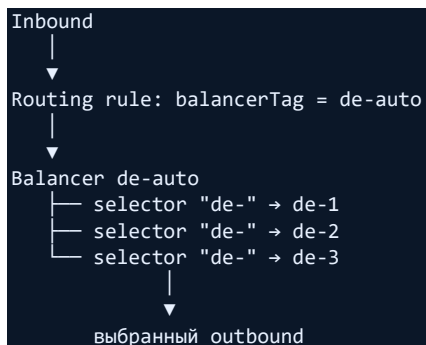
- URL;
- `tagPrefix` — безопасный префикс tags;
- update interval, типично от 600 секунд;
- prepend — ставить импортированные записи раньше;
- priority — порядок обработки.

После обновления проверьте, что selector balancer всё ещё выбирает ожидаемые tags, а неизвестные/несовместимые поля не были отброшены core.

8. Balancers — подробное руководство

8.1. Что делает Xray Balancer

Balancer выбирает **один outbound для нового соединения** из группы tags. Он не распределяет входящие подключения между экземплярами Зх-ui и не объединяет пропускную способность нескольких каналов в один TCP-сеанс.



То есть это **outbound egress balancer внутри Xray**, а не AWS/HAProxy/Nginx перед сервером.

8.2. Поля Balancer

Поле	Назначение
tag	Уникальное имя balancer; на него ссылается rule balancerTag
selector	Список префиксов outbound tags
strategy	random, roundRobin, leastPing, leastLoad
fallbackTag	Outbound или другой balancer для случая недоступности основной группы

Selector — префиксное сопоставление. `de-` выберет `de-1`, `de-backup` и `de-test`; поэтому имена должны исключать случайные совпадения.

8.3. Стратегии

Random

Выбирает случайный подходящий outbound для новых соединений. Прост, не требует измерять latency, если fallback не используется. Это статистическое распределение, а не гарантия равной нагрузки.

Round Robin

Последовательно перебирает outbounds. Удобен для примерно одинаковых узлов и предсказуемого распределения количества соединений. Он не учитывает размер/длительность потоков: один узел может получить несколько тяжёлых сеансов.

Least Ping

Использует результаты Observatory и выбирает outbound с минимальным измеренным RTT/успешным probe. Низкий ping до test URL не гарантирует лучшую скорость до каждого реального сайта, но хорошо решает выбор ближайшего здорового выхода.

Least Load

Использует Burst Observatory и оценку нагрузки/RTT. Дополнительные поля:

Поле	Назначение
<code>expected</code>	Ожидаемое число лучших кандидатов в выборке
<code>maxRTT</code>	Отсечь выходы с RTT выше порога
<code>tolerance</code>	Допуск 0–1 к различию оценки, чтобы не дёргать выбор при малой разнице
<code>baselines</code>	Окна времени для базовой оценки
<code>costs.regex / match</code>	Какие tags/результаты получают поправку
<code>costs.value</code>	Стоимость; большее значение делает кандидат менее предпочтительным

Costs позволяют, например, предпочитать собственный выход платному резерву при сопоставимом состоянии. Неправильные regex могут затронуть лишние tags.

8.4. Observatory и Burst Observatory

Observatory

- `subjectSelector` — префиксы наблюдаемых outbound tags;
- `probeURL` — URL проверки;
- `probeInterval` — период проверки;
- `enableConcurrency` — проверять параллельно.

Burst Observatory

- `subjectSelector`;
- `pingConfig.destination` — адрес проверки;
- `connectivity` — режим проверки доступности;

- `interval`, `timeout`, `sampling`;
- `httpMethod` — HEAD или GET.

3x-ui v3.6.0 управляет observers автоматически:

- `leastPing` использует обычный Observatory, если Burst не требуется другим balancer;
- `leastLoad` использует Burst Observatory;
- `random/roundRobin` с `fallbackTag` также требуют наблюдения доступности и в логике панели подключают Burst;
- если нужен Burst, панель объединяет в него selector-ы и не держит конкурирующий обычный Observatory.

Модель и интервалы probe описаны в [Observatory Xray](#).

8.5. Fallback

`fallbackTag` используется, когда основная группа не даёт пригодного результата. Он может указывать на outbound или другой balancer. Для balancer→balancer 3x-ui создаёт внутреннюю loopback-связку и проверяет циклы.

Пример:

```
eu-primary: selector eu-paid-, leastPing
            fallbackTag → eu-backup
eu-backup: selector eu-spare-, roundRobin
```

Fallback — не повтор каждого неудачного пользовательского запроса на всех узлах. Доступность определяется observers и состоянием, а уже затем выбирается маршрут нового соединения.

8.6. Правильная последовательность настройки

1. Создать и вручную проверить каждый outbound.
2. Дать tags однозначные префиксы: `de-prod-1`, `de-prod-2`.
3. Выбрать test URL, который стабильно отвечает извне и отражает реальный маршрут; ответ 204 удобен из-за отсутствия body.
4. Создать balancer и selector `de-prod-`.
5. Для heterogeneous-сети выбрать `leastPing/leastLoad`; для одинаковых узлов — `roundRobin/random`.
6. Создать Routing Rule с `balancerTag`, не `outboundTag`.
7. Проверить logs/observatory status и фактический внешний IP.
8. Добавить fallback только после проверки основной группы.

8.7. Какие задачи Balancer решает

- автоматический выбор живого egress;
- распределение новых соединений между несколькими провайдерами;
- выбор низкой задержки;
- предпочтение дешёвого/собственного выхода через costs;
- региональные пулы: отдельный balancer для DE, NL, US;
- обслуживание без остановки: вывести tag из selector и дождаться завершения старых сессий;
- использование импортированного subscription pool с устойчивым tagPrefix.

8.8. Чего Balancer не делает

- не скрывает IP inbound VPS;
- не балансирует пользователей между несколькими 3x-ui серверами;
- не ускоряет один TCP/QUIC-сеанс сложением каналов;
- не синхронизирует state между backend;
- не заменяет DNS, BGP anycast, L4/L7 load balancer или CDN;
- не гарантирует, что низкий RTT означает отсутствие блокировки конкретного сайта.

8.9. Ошибки и диагностика

Симптом	Проверить
Пул пуст	Selector и реальные tags, регистр и prefix collision
Всегда один outbound	Стратегию, observer status, costs и текущие долгие соединения
Fallback не включается	Probe URL, timeout/interval, тип fallbackTag
Слишком много фонового трафика	Частоту probes, GET вместо HEAD, число outbounds
Core не запускается	Цикл balancers, отсутствующий tag, несовместимый observer JSON
IP не меняется на каждом запросе	Keep-alive/mux: несколько запросов идут в одном уже выбранном соединении

9. DNS в Xray Configs

Встроенный DNS нужен не для «ускорения Интернета вообще», а для согласованного разрешения доменов внутри Xray, работы domainStrategy, геомаршрутизации и FakeDNS. Актуальная логика серверов описана в [DNS Xray](#).

9.1. Глобальные поля

Поле	Назначение
Enable	Включить DNS-модуль шаблона
tag	Tag DNS-модуля/трафика для routing
clientIp	EDNS Client Subnet, адрес подсети клиента для geo-ответа
queryStrategy	UseIP, UseIPv4, UseIPv6, UseSystem
disableCache	Не кешировать ответы
disableFallback	Не обращаться к fallback-серверам
disableFallbackIfMatch	Если сервер выбран по domain rule, не делать fallback
enableParallelQuery	Запрашивать подходящие серверы параллельно
useSystemHosts	Учитывать hosts ОС
serveStale	Возвращать устаревший кеш при проблеме обновления
serveExpiredTTL	Сколько ещё обслуживать истёкшую запись

[clientIp](#) может улучшить географический ответ CDN, но раскрывает резолверу дополнительную информацию о подсети. [disableCache](#) увеличивает задержку и число запросов. [serveStale](#) повышает устойчивость ценой временно устаревших адресов.

9.2. Hosts

Hosts — локальная таблица [domain](#) → [address/domain](#). Она применяется до внешнего resolver и подходит для:

- фиксированного внутреннего имени;
- тестового перенаправления;
- блокировки известным sink address;
- CNAME-подобной подстановки другого домена.

Не путать DNS Hosts с отдельной страницей Зх-ui **Hosts**, заменившей legacy External Proxy: название похоже, слой и назначение различаются.

9.3. Поля DNS server

Поле	Назначение
address	localhost, IP, tcp://, tcp+local://, https://, https+local://, quic+local://, fakedns
port	Порт для схем, где он задаётся отдельно
tag	Tag запросов этого resolver
clientIP	ECS только для этого сервера
queryStrategy	IPv4/IPv6/system стратегия сервера
timeoutMs	Таймаут, UI по умолчанию около 4000 мс
domains	Для каких доменов сервер приоритетен
expectedIPs	Допустимые IP/CIDR/geoip результата
unexpectedIPs	Результаты, считаемые неприемлемыми
skipFallback	Не включать сервер в общий fallback
finalQuery	После выбора этого сервера не продолжать поиск
disableCache	Не кешировать его ответы
serveStale, serveExpiredTTL	Политика просроченного кеша

Суффикс `+local` означает локальное соединение resolver, минуя обычную Xray routing-схему; это может создать DNS leak, если ожидалось проксирование. Encrypted DNS защищает участок до resolver, но сам resolver всё равно знает запрошенное имя.

`expectedIPs/unexpectedIPs` позволяют признать ответ неподходящим и перейти к следующему resolver. `finalQuery` останавливает такой перебор. Порядок DNS servers и доменные filters поэтому важны.

9.4. FakeDNS

- `ipPool` — виртуальная сеть адресов FakeDNS;
- `poolSize` — размер пула.

FakeDNS выдаёт приложению искусственный IP и сохраняет связь IP→domain, чтобы Xray восстановил домен при прозрачном/TUN-сценарии. Пул не должен конфликтовать с реальными локальными маршрутами. FakeDNS без перехвата трафика и sniffing `fakedns` не решает задачу сам по себе.

9.5. DNS-проверка

1. Убедиться, где физически выполняется DNS-запрос: клиент, сервер, local resolver или proxy outbound.
 2. Проверить A и AAAA отдельно.
 3. Сверить Routing Strategy и outbound domainStrategy.
 4. Проверить домен с включённым и выключенным Sniffing.
 5. Проверить утечки через системный resolver.
 6. Учитывать ECH/HTTPS records, а не только A/AAAA.
-

10. Advanced

Advanced предоставляет прямое редактирование частей JSON:

- `completeTemplate` — весь глобальный Xray template;
- `inbounds` — raw массив дополнительных/служебных inbounds;
- `outbounds` — raw массив outbounds;
- `routing.rules` — raw rules.

Использовать Advanced следует, только если визуальный редактор не поддерживает стабильное поле текущего core. Панель добавляет собственные API/stats/metrics objects и преобразует часть tags; ручной JSON может конфликтовать с генератором.

10.1. Безопасный порядок работы

1. Экспортировать backup Зх-ui.
2. Сохранить исходный template отдельно.
3. Внести одно логическое изменение.
4. Проверить JSON и ссылки на tags.
5. Перезапустить core и проверить status/error log.
6. Проверить inbound, DNS и каждый затронутый outbound.
7. Только затем переносить изменение на другие nodes.

Не вставляйте в публичный issue полный template: он может содержать UUID, passwords, private keys, ECH/REALITY secrets, subscription URLs и адреса инфраструктуры.

11. IPv6, Anycast, rotating IP, load balancers и cloud edge

11.1. Что именно можно скрыть

```
Прямой VPS
Client → Origin
ISP клиента видит IP Origin

Через edge
Client → CDN / L4 edge / relay → Origin
ISP клиента видит IP edge
Edge-провайдер знает Origin
```

IP не исчезает; меняется участник, адрес которого виден на конкретном участке. Защитить origin означает одновременно:

- не публиковать его в клиентской ссылке/DNS;
- разрешить firewall origin принимать проху-порт только от edge/relay;
- исключить утечки через старые DNS records, панели, почту, другие vhosts и IPv6;
- защитить административную панель отдельно.

11.2. IPv6

IPv6 использует 128-битные адреса и даёт каждому интерфейсу глобальную адресацию без обязательного NAT. Для Зх-ui это означает:

- bind `::` и осознанный `v6only`;
- AAAA record наряду с A;
- отдельные IPv6 firewall/security-group rules;
- корректные outbound `UseIPv6/ForceIPv6`/dual-stack strategies;
- Happy Eyeballs при dual stack;
- квадратные скобки вокруг literal IPv6 в URL: `https://[2001:db8::1]:443/`.

IPv6 не маскирует сервер. Провайдер видит destination IPv6 так же, как IPv4. Он иногда обходит IPv4-only блокировку, но может создать незамеченный прямой путь к origin, если CDN/firewall настроены только для A/IPv4.

11.3. Anycast

Anycast — один и тот же IP-префикс объявляется из нескольких точек сети. BGP доставляет клиента к выбранной по политике маршрутизации точке; это не обязательно географически ближайший сервер. Архитектурные ограничения описаны в [RFC 4786](#).

Свойства:

- видимый IP остаётся стабильным;
- ingress распределяется сетью, до Xray;
- failover возможен через изъятие BGP route/health logic;
- stateful TCP/QUIC требует, чтобы маршрут не «прыгал» между несинхронизированными POP во время сессии;
- собственный Anycast требует BGP/ASN/IP-prefix или услуги провайдера; это не переключатель Зх-ui.

Anycast скрывает отдельный backend за сервисным адресом, но сам anycast IP прекрасно виден и может быть заблокирован.

11.4. Rotating IP

Rotating IP — периодическая смена публикуемого ingress или egress address.

Варианты:

- менять A/AAAA records и серверы;
- выдавать пользователям новые endpoints через subscription;
- выбирать разные egress outbounds;
- получать меняющийся egress от внешнего proxy/WARP-подобного сервиса.

Ограничения:

- текущая TCP/QUIC-сессия не переносится на новый IP автоматически;
- DNS TTL и кеш resolver задерживают ротацию;
- сертификат/SNI и ссылки должны оставаться валидными;
- слишком частая смена создаёт сбои и аномальный профиль;
- egress rotation не меняет адрес inbound, который видит ISP клиента;
- старый origin надо закрывать, иначе ротация лишь добавляет адреса.

Ротация полезна для замены заблокированного/перегруженного адреса и распределения egress, но не создаёт анонимность сама по себе.

11.5. Четыре разных вида балансировки

Механизм	Где работает	Что выбирает	Скрывает origin от ISP клиента
Xray Balancer	После inbound, внутри Xray	Исходящий outbound для нового соединения	Нет
Внешний L4/L7 load balancer	Перед Зх-ui/Xray backend	Backend server	Да, если direct доступ закрыт

Механизм	Где работает	Что выбирает	Скрывает origin от ISP клиента
DNS round-robin/geo DNS	До соединения	IP, возвращаемый клиенту	Только если возвращаются edge, а не origins
Anycast/cloud edge	Сетевая граница провайдера	POP/edge по маршрутизации и health	Да, backend не виден клиентскому участку

L4 load balancer

Балансирует TCP/UDP без понимания внутреннего HTTP. Может переносить RAW TCP и UDP/QUIC, если сервис провайдера поддерживает нужный протокол и сохраняет state. TLS часто проходит passthrough до Xray.

L7 reverse proxy/CDN

Понимает HTTP Host/path/headers и обычно завершает TLS на edge. Подходит для WebSocket, HTTPUpgrade, XHTTP и, при поддержке, gRPC. Прямой VLESS RAW + REALITY нельзя просто поместить за обычный HTTP CDN: CDN не передаёт произвольное REALITY-рукопожатие как HTTP request.

DNS balancing

Возвращает несколько A/AAAA или разные адреса по географии. Клиент/recursive resolver кеширует выбор; DNS сам не проверяет прикладное состояние каждой уже начатой сессии и публикует выбранные IP.

11.6. Cloud edge

Cloud edge — POP провайдера, который становится первой точкой соединения. Он может быть:

- CDN/reverse proxy уровня HTTP;
- anycast L4 accelerator;
- защищённый relay;
- глобальный load balancer с health checks.

Например, AWS Global Accelerator выдаёт статические anycast IP и направляет трафик к здоровым regional endpoints; выбор endpoints и health checks описан в [официальной документации AWS](#).

Cloud edge решает origin shielding, DDoS absorption, близкий ingress и failover, но:

- edge-провайдер видит origin и метаданные участка;
- совместимость зависит от L4/L7 и поддерживаемых TCP/UDP/HTTP функций;
- TLS termination определяет, кто видит plaintext транспортного HTTP;
- backend firewall должен принимать только адреса/аутентифицированные tunnels edge;

- логирование real client IP требует доверенного PROXY protocol/header и защиты от spoofing.

11.7. Рабочие архитектуры

Простая и производительная

Client → VLESS + RAW + REALITY → один VPS → direct/balancer egress

Плюсы: минимум компонентов. Минус: ISP клиента видит VPS IP; origin не скрыт.

CDN-совместимая

Client → TLS → CDN → WebSocket/XHTTP/HTTPUpgrade → Xray origin

Плюсы: ISP видит edge IP, origin можно закрыть firewall. Минусы: совместимость транспорта, правила CDN, HTTP overhead, доверие провайдеру.

L4 relay/accelerator

Client → anycast/L4 edge → RAW TCP или UDP/QUIC → Xray backend

Плюсы: сохраняет больше транспортов. Минусы: стоимость, stateful routing, поддержка UDP и PROXY protocol.

Два управляемых сервера

Client → Front Xray → защищённый outbound → Backend Xray → Internet

Плюсы: backend скрыт от клиента/его ISP. Минусы: дополнительная задержка, front остаётся видимым, две точки эксплуатации.

12. Xray и реально используемые альтернативные ядра

12.1. Роль Xray-core

Xray-core — основной data plane 3x-ui: listeners, VLESS/VMess/Trojan/Hysteria/WireGuard proxy, transport security, sniffing, DNS, routing, outbounds, observatory и balancers. Панель не «является Xray» и не шифрует трафик сама.

3x-ui v3.6.0 собирает конфигурации под Xray-core v26.7.28. Поставить другое core технически возможно только с учётом формата JSON и функций; несовпадение версии особенно критично для новых XHTTP session fields, VLESS Encryption, REALITY ML-DSA и FinalMask.

12.2. Другие применяемые проекты

- **v2ray-core** — предшествующая/родственная экосистема, но без полного набора новых Xray-specific функций.
- **sing-box** — самостоятельное универсальное прокси-ядро; популярно на клиентах и серверах, но названия/семантика полей не тождественны Xray.
- **Mihomo** — ядро Clash-экосистемы для rule-based клиентов; поддерживает многие links, но совместимость новых Xray features может отставать.
- **native WireGuard** — отдельный VPN-интерфейс ОС, если нужен чистый L3 tunnel без Xray orchestration.
- **официальный Hysteria2 core** — специализированная реализация Hysteria2.
- **mtg-multi** — фактически используется 3x-ui для MTProto и управляется рядом с Xray.

3x-ui не предоставляет универсальный переключатель «заменить Xray на sing-box/Mihomo». Наличие протокола в клиенте ещё не означает поддержку всех security/mask/auth полей конкретной версии сервера.

13. Практические профили и проверка

13.1. VLESS + REALITY для прямого VPS

1. VLESS, RAW, REALITY.
2. UUID клиента; `xtls-rprx-vision` — основной flow при поддержке клиента.
3. `xtls-rprx-vision-udp443` выбирать только если приложению действительно нужен QUIC и оно не делает TCP fallback.
4. Убедиться, что клиентская ссылка содержит выбранный flow, а серверная runtime-запись содержит базовый `xtls-rprx-vision`.
5. Target и serverNames проверить scanner-ом.
6. Сгенерировать REALITY key pair и short ID.
7. Выбрать распространённый uTLS fingerprint.
8. Обычный Mux выключить.
9. Sniffing включить только если нужны domain rules; начать с routeOnly.
10. Не ожидать скрытия IP VPS.

13.2. ХНТТР/WS через CDN

1. Создать hostname и сертификат на edge.
2. Выбрать транспорт, поддерживаемый CDN.
3. Согласовать Host/path/ALPN на клиенте, edge и origin.
4. На origin разрешить соединения только от edge/relay.
5. Настроить real IP только через доверенный header/PROXY protocol.
6. Проверить, что прямой IPv4 и IPv6 origin недоступны.

13.3. Hysteria2 с резервом

1. Проверить UDP на клиентской сети.
2. Валидный TLS certificate/SNI/ALPN.
3. Уникальный auth и Salamander password.
4. Начать с стандартного congestion profile; не завышать Brutal speed.
5. UDP Нор включать только после открытия и маршрутизации полного диапазона.
6. Отдельно выдать TCP-профиль на случай блокировки UDP.

13.4. Outbound balancer

1. Проверить каждый outbound без balancer.
2. Префиксы tags без пересечений.
3. Probe URL и reasonable interval.
4. Rule с balancerTag.
5. Проверить новый внешний IP в новых соединениях без keep-alive/mux.
6. Смоделировать отказ одного выхода и возврат после восстановления.

13.5. Общий чек-лист перед публикацией подключения

- Port открыт только там, где требуется.
- Panel/API не опубликованы на том же незащищённом endpoint.
- UUID/password/auth/private keys уникальны.
- Client и server поддерживают все выбранные новые поля.
- Transport, Host/path/serviceName и security совпадают.
- SNI соответствует certificate либо REALITY target/serverNames.
- ALPN соответствует transport/proxy.
- PROXY protocol включён или выключен на обеих сторонах участка.
- Sniffing и routeOnly проверены на реальных приложениях.
- Routing rules расположены от узких к широким.
- Default outbound известен.
- А и AAAA проверены; нет обходного прямого origin path.
- DNS не утекает мимо предполагаемого маршрута.
- Logs не хранят лишние IP/domains и ротируются.
- Backup конфигурации сделан без публикации secrets.

14. Быстрый указатель полей со скриншотов

Этот раздел нужен как карта поиска по справочнику.

Группа со скриншотов	Где разобрано
REALITY: Xver, Target, SNI, Max Time Diff, client versions, Short IDs, SpiderX, keys, ML-DSA	3.7
Sniffing: destOverride, metadataOnly, routeOnly, exclusions	4
DPI: видимые признаки, классификация VLESS/Trojan/Hysteria2/Shadowsocks/WireGuard	5
Hysteria2: QUIC/BBR/Brutal/UDP Hop/windows	6.3 и 6.9
TLS: Reject Unknown SNI, Session Resumption, certificates, OCSP, ECH, pinned SHA-256	3.6
VLESS Encryption/Auth и testseed	2.2 и 6.3
VLESS flow, XTLS Vision, Vision UDP/443, Mux и legacy XTLS	2.3
RAW, Proxy Protocol, HTTP camouflage	6.4–6.5
External Proxy	6.12: перенесён в Hosts
Sockopt	6.10
TCP/UDP Masks, Salamander	6.11
Xray Configs Basics/Routing/Outbounds/DNS/Advanced	7, 9, 10
Balancers	8
IPv6/Anycast/rotating IP/load balancers/cloud edge	11

15. Термины, которые нельзя смешивать

Термин	Правильный короткий смысл
Inbound	Где Xray принимает клиента
Outbound	Куда и как Xray устанавливает следующий участок
Proxy protocol	VLESS/Trojan/VMess и т. п.; не путать с HAProxy PROXY protocol
PROXY protocol	Служебный заголовок v1/v2 с исходным IP между доверенными проху
Transport	RAW/WS/gRPC/XHTTP/HTTPUpgrade/mKCP/Hysteria
TLS	Шифрование и проверка transport endpoint
REALITY	Модифицированная TLS transport security Xray с target
SNI	Имя хоста в TLS ClientHello
ECH	Шифрование внутреннего ClientHello, не IP
ALPN	Согласование HTTP/2, HTTP/1.1, HTTP/3 и т. п.
uTLS	Имитация формы ClientHello
DPI	Stateful-анализ пакетов и потоков по содержимому, handshake, метаданным и статистике
Sniffing	Извлечение доступного домена/метаданных после inbound
FinalMask	Дополнительное оформление transport, не обязательно шифрование
Xray Balancer	Выбор egress outbound
External load balancer	Выбор inbound backend перед Xray
Anycast	Сетевая доставка одного IP к одному из POP
CDN/cloud edge	Посредник на edge; L7 или L4 в зависимости от продукта

16. Основные источники

3x-ui v3.6.0

- [Release v3.6.0](#)
- [Release workflow v3.6.0 — версия комплектного Xray-core](#)
- [Inbounds](#)
- [Clients](#)
- [Transports](#)
- [REALITY](#)
- [Outbounds and routing](#)
- [Генерация runtime client flow в 3x-ui v3.6.0](#)

Xray-core

- [Configuration overview](#)
- [VLESS inbound и серверный flow](#)
- [VLESS outbound, Vision, UDP/443 и splice](#)
- [Допустимые значения flow в Xray-core v26.7.28](#)
- [Transport](#)
- [TLS](#)
- [REALITY](#)
- [Inbound and Sniffing](#)
- [Routing and Balancers](#)
- [DNS](#)
- [Observatory](#)

DPI и анализ зашифрованного трафика

- [RFC 8446: TLS 1.3 — traffic analysis по длинам и времени](#)
- [RFC 9000 и RFC 9001: QUIC и TLS для QUIC](#)
- [RFC 9312: наблюдаемость и управление QUIC](#)
- [RFC 9849: Encrypted ClientHello и его границы](#)
- [USENIX Security 2023: пассивное обнаружение полностью зашифрованного проху-трафика](#)
- [FOCI 2020: active probing и probe-resistant proxies](#)

Сетевая инфраструктура

- [RFC 4786: Operation of Anycast Services](#)
- [RFC 4291: IPv6 Addressing Architecture](#)
- [AWS Global Accelerator: how it works](#)
- [AWS Global Accelerator endpoints](#)